МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ (НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №2

по курсу «Программирование графических процессоров»

Обработка изображений на GPU. Фильтры

Выполнил: Л.А. Фокин

Группа: М8О-401Б-23

Преподаватель: А.Ю. Морозов

Москва, 2026

Содержание

1 Условие **2**
2 Программное и аппаратное обеспечение **2**
3 Метод решения **3**
4 Описание программы **4**
5 Результаты **7**
6 Выводы **9**

1 Условие

Цель работы: реализовать обработку изображения на GPU с использованием текстурной памяти и двумерной сетки потоков и сравнить время работы с реализацией на CPU.

Вариант 7. Выделение контуров методом Собеля

На первой строке задаётся путь к исходному изображению, на второй — путь к результату. Формат файла: два четырёхбайтовых целых числа w и h, затем пиксели RGBA по четыре байта, построчно. Ограничения: $w, h < 2^{16}$, $wh \leq 10^8$. Используются цветовые каналы RGB, альфа-канал не учитывается. При выходе за границы изображения используется ближайший граничный пиксель. Допустимая погрешность цветового канала составляет ± 1.

2 Программное и аппаратное обеспечение

Аппаратное обеспечение

- **Графический процессор (GPU):** NVIDIA GeForce RTX 5070
  - Compute Capability: 12.0
  - Объем глобальной памяти (Global Memory): 12 792 МБ ($\approx$ 12 ГБ) GDDR7
  - Объем разделяемой памяти на блок: 49 152 байт
  - Объем константной памяти: 65 536 байт
  - Количество 32-битных регистров на блок: 65 536
  - Количество потоковых мультипроцессоров (SM): 48
  - Максимальное количество нитей на блок: 1024
  - Максимальный размер блока: $(1024, 1024, 64)$
  - Максимальный размер сетки: $(2147483647, 65535, 65535)$
- **Центральный процессор (CPU):** AMD Ryzen 5 8400F 6-Core Processor (6 физических ядер, 12 логических потоков, базовая тактовая частота 4.2 ГГц, Max Boost до 4.7 ГГц, кэш L3 16 МБ)
- **Оперативная память (RAM):** 16 ГБ DDR5
- **Жесткий диск (Накопитель):** NVMe Solid State Drive (SSD) емкостью 1 ТБ, файловая система ext4

Программное обеспечение

- **Операционная система (OS):** Ubuntu 24.04 LTS (x86_64 Linux kernel 6.6)
- **Среда разработки (IDE / Редактор):** Zed Editor / Visual Studio Code
- **Компилятор CUDA:** NVIDIA CUDA Compiler Driver (nvcc) v12.0 (V12.0.140)
- **Компилятор C++ хоста:** g++ (Ubuntu 13.3.0)

3 Метод решения

Яркость каждого пикселя вычисляется по формуле $Y = 0{,}299R + 0{,}587G + 0{,}114B$. Для соседства 3×3 применяются две маски Собеля:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * Y, \qquad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} * Y.$$

Значение выходного пикселя равно $\min(255, \sqrt{G_x^2 + G_y^2})$ и записывается в три цветовых канала. Альфа-канал результата равен нулю.

Исходное изображение хранится в двумерной текстуре CUDA с точечной фильтрацией и режимом адресации Clamp. При выходе за границы текстура возвращает ближайший граничный пиксель. Потоки организованы в двумерную сетку фиксированного размера. Каждый поток обрабатывает пиксели с шагом, равным размеру сетки по соответствующей оси. Время алгоритма составляет $O(wh)$, объём памяти — $O(wh)$.

Источники и материалы

1. NVIDIA Corporation. *CUDA C++ Programming Guide (Release 12.x)*. URL: `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`.

2. Морозов А. Ю. *Материалы курса по программированию графических процессоров (CUDA)*: лекции «Программные возможности CUDA», «Память CUDA», «Фильтры обработки изображений»; пример работы с текстурами `Sample Texture.ipynb`; архив «Бинарные данные из условия 2, 3 и 5 ЛР». URL: `https://drive.google.com/drive/folders/1XqbrmRt70LJOwScFF4dptp5WR2sQnO-8`.

3. Методические указания к лабораторным работам по курсу «Программирование графических процессоров»: *Лабораторная работа №2. Обработка изображений на GPU. Фильтры.* МАИ, 2024.

4 Описание программы

Файл `main.cu` содержит ядро `sobel`, функции ввода и вывода бинарного изображения и создания текстурного объекта. Ядро считывает соседние пиксели из текстурной памяти, вычисляет яркость и градиент, затем записывает результат. Ошибки CUDA и файлового ввода-вывода выводятся в `stderr`. Конфигурацию сетки для замеров можно задать аргументами командной строки.

Файл `cpu_ref.cpp` содержит однопоточную реализацию: сначала вычисляются яркости всех пикселей, затем свёртки по обеим маскам и величина градиента. Конвертер `converter.py` переводит изображения между обычными форматами и бинарным форматом задания.

Исходный код программы (`main.cu`)

```
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <ctime>

#include <cuda_runtime.h>

#define CSC(call)                                                              \
do {                                                                           \
    cudaError_t res = call;                                                    \
    if (res != cudaSuccess) {                                                  \
        std::fprintf(stderr, "ERROR: %s:%d: %s\n",                             \
                     __FILE__, __LINE__, cudaGetErrorString(res));             \
        std::exit(0);                                                          \
    }                                                                          \
} while (0)


__device__ float brightness(cudaTextureObject_t tex, int x, int y) {
    uchar4 p = tex2D<uchar4>(tex, x + 0.5f, y + 0.5f);
    return 0.299f * p.x + 0.587f * p.y + 0.114f * p.z;
}

__global__ void sobel(cudaTextureObject_t tex, uchar4* dst, int w, int h) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int idy = blockIdx.y * blockDim.y + threadIdx.y;
    int offx = blockDim.x * gridDim.x;
    int offy = blockDim.y * gridDim.y;
    for (int y = idy; y < h; y += offy) {
        for (int x = idx; x < w; x += offx) {
            float p00 = brightness(tex, x - 1, y - 1);
            float p01 = brightness(tex, x, y - 1);
            float p02 = brightness(tex, x + 1, y - 1);
            float p10 = brightness(tex, x - 1, y);
            float p12 = brightness(tex, x + 1, y);
            float p20 = brightness(tex, x - 1, y + 1);
            float p21 = brightness(tex, x, y + 1);
            float p22 = brightness(tex, x + 1, y + 1);
            float gx = (p02 + 2.0f * p12 + p22) - (p00 + 2.0f * p10 + p20);
            float gy = (p20 + 2.0f * p21 + p22) - (p00 + 2.0f * p01 + p02);
            unsigned char value = (unsigned char)fminf(255.0f, sqrtf(gx * gx + gy * gy));
            dst[y * w + x] = make_uchar4(value, value, value, 0);
        }
    }
}

void read_image(const char* path, int& w, int& h, uchar4*& data) {
    FILE* fp = fopen(path, "rb");
    if (!fp) {
        std::fprintf(stderr, "ERROR: cannot open %s\n", path);
        std::exit(0);
    }
    if (fread(&w, sizeof(int), 1, fp) < 1 || fread(&h, sizeof(int), 1, fp) < 1) {
        std::fprintf(stderr, "ERROR: broken image file %s\n", path);
        std::exit(0);
```

```
    }
    if (w <= 0 || h <= 0 || w >= 65536 || h >= 65536 || (size_t)w * h > 100000000) {
        std::fprintf(stderr, "ERROR: invalid image dimensions\n");
        std::exit(0);
    }
    data = (uchar4*)std::malloc(sizeof(uchar4) * w * h);
    if (!data) {
        std::fprintf(stderr, "ERROR: cannot allocate image\n");
        std::exit(0);
    }
    if (fread(data, sizeof(uchar4), (size_t)w * h, fp) < (size_t)w * h) {
        std::fprintf(stderr, "ERROR: broken image file %s\n", path);
        std::exit(0);
    }
    fclose(fp);
}

void write_image(const char* path, int w, int h, uchar4* data) {
    FILE* fp = fopen(path, "wb");
    if (!fp) {
        std::fprintf(stderr, "ERROR: cannot open %s for writing\n", path);
        std::exit(0);
    }
    bool ok = fwrite(&w, sizeof(int), 1, fp) == 1 &&
              fwrite(&h, sizeof(int), 1, fp) == 1 &&
              fwrite(data, sizeof(uchar4), (size_t)w * h, fp) == (size_t)w * h;
    if (fclose(fp) != 0) ok = false;
    if (!ok) {
        std::fprintf(stderr, "ERROR: cannot write %s\n", path);
        std::exit(0);
    }
}

void make_object_uchar4(uchar4* host, size_t w, size_t h,
                        cudaArray_t& arr, cudaTextureObject_t& tex) {
    cudaChannelFormatDesc desc = cudaCreateChannelDesc<uchar4>();
    CSC(cudaMallocArray(&arr, &desc, w, h));
    CSC(cudaMemcpy2DToArray(arr, 0, 0, host, sizeof(uchar4) * w,
                            sizeof(uchar4) * w, h, cudaMemcpyHostToDevice));

    cudaResourceDesc rd = {};
    rd.resType = cudaResourceTypeArray;
    rd.res.array.array = arr;

    cudaTextureDesc td = {};
    td.addressMode[0] = cudaAddressModeClamp;
    td.addressMode[1] = cudaAddressModeClamp;
    td.filterMode = cudaFilterModePoint;
    td.normalizedCoords = false;

    CSC(cudaCreateTextureObject(&tex, &rd, &td, nullptr));
}

int main(int argc, char** argv) {
    char path_in[1024], path_out[1024];
    if (!fgets(path_in, sizeof(path_in), stdin) || !fgets(path_out, sizeof(path_out), stdin)
    ) {
        std::fprintf(stderr, "ERROR: expected input and output paths\n");
        return 0;
    }
    for (char* s : {path_in, path_out}) {
        for (int i = 0; s[i]; ++i) {
            if (s[i] == '\n' || s[i] == '\r') s[i] = '\0';
        }
    }
    int bx = 32, by = 32, tx = 16, ty = 16;
    if (argc != 1) {
        char extra;
        if (argc != 5 || std::sscanf(argv[1], "%d%c", &bx, &extra) != 1 ||
            std::sscanf(argv[2], "%d%c", &by, &extra) != 1 ||
            std::sscanf(argv[3], "%d%c", &tx, &extra) != 1 ||
            std::sscanf(argv[4], "%d%c", &ty, &extra) != 1 ||
            bx < 1 || by < 1 || tx < 1 || ty < 1) {
            std::fprintf(stderr, "ERROR: invalid launch configuration\n");
            return 0;
        }
```

```
    }
    int w, h;
    uchar4* img = nullptr;
    read_image(path_in, w, h, img);
    cudaArray_t arr = nullptr;
    cudaTextureObject_t tex;
    make_object_uchar4(img, w, h, arr, tex);
    uchar4* out = nullptr;
    CSC(cudaMalloc(&out, sizeof(uchar4) * w * h));
    cudaEvent_t start, stop;
    CSC(cudaEventCreate(&start));
    CSC(cudaEventCreate(&stop));
    CSC(cudaEventRecord(start));
    sobel<<<dim3(bx, by), dim3(tx, ty)>>>(tex, out, w, h);
    CSC(cudaGetLastError());
    CSC(cudaEventRecord(stop));
    CSC(cudaEventSynchronize(stop));
    float ms;
    CSC(cudaEventElapsedTime(&ms, start, stop));
    std::fprintf(stderr, "kernel_ms = %.6f\n", ms);
    CSC(cudaMemcpy(img, out, sizeof(uchar4) * w * h, cudaMemcpyDeviceToHost));
    write_image(path_out, w, h, img);
    CSC(cudaEventDestroy(start));
    CSC(cudaEventDestroy(stop));
    CSC(cudaDestroyTextureObject(tex));
    CSC(cudaFreeArray(arr));
    CSC(cudaFree(out));
    std::free(img);
    return 0;
}
```

5 Результаты

Для проверки использованы изображения `97.data` и `95.data` из архива бинарных данных курса. Результаты на обоих примерах варианта 7 совпали с условием с допустимой погрешностью ± 1 по каждому каналу.

Замеры проведены на одноцветных синтетических изображениях размером 500×200, 2500×2000 и 10000×10000 пикселей. Последний размер соответствует ограничению в 10^8 пикселей. CPU-программа собрана с `g++ -O3`. В таблице приведено время CPU-вычислений и время ядра GPU, измеренное CUDA Events. Для GPU не учитываются передача данных и файловый ввод-вывод. Приведён один запуск для каждой конфигурации, поэтому результаты зависят от нагрузки системы. Размер блока GPU — 16×16 нитей; меняются размеры сетки (B_x, B_y). Время указано в миллисекундах.

Дополнительно для изображения 500×200 проверены конфигурации $\langle\langle\langle 1, 32 \rangle\rangle\rangle$ и $\langle\langle\langle 1024, 1024 \rangle\rangle\rangle$ с одномерным расположением блоков и нитей: время ядра составило 1,01 и 0,33 мс соответственно.

Таблица 1: Время работы CPU и GPU

Пикселей	Сетка (B_x, B_y)	CPU, мс	GPU, мс
100 000	$(1, 32)$	1.34	0.171
100 000	$(1, 256)$	1.34	0.237
100 000	$(1, 1024)$	1.34	0.156
100 000	$(32, 256)$	1.34	0.155
100 000	$(256, 256)$	1.34	0.187
100 000	$(1024, 256)$	1.34	0.403
100 000	$(1024, 1024)$	1.34	0.790
5 000 000	$(1, 32)$	48.00	0.469
5 000 000	$(1, 256)$	48.00	0.314
5 000 000	$(1, 1024)$	48.00	0.402
5 000 000	$(32, 256)$	48.00	0.317
5 000 000	$(256, 256)$	48.00	0.607
5 000 000	$(1024, 256)$	48.00	0.471
5 000 000	$(1024, 1024)$	48.00	1.507
100 000 000	$(1, 32)$	1024.48	7.617
100 000 000	$(1, 256)$	1024.48	2.238
100 000 000	$(1, 1024)$	1024.48	2.047
100 000 000	$(32, 256)$	1024.48	4.316
100 000 000	$(256, 256)$	1024.48	1.887
100 000 000	$(1024, 256)$	1024.48	1.974
100 000 000	$(1024, 1024)$	1024.48	3.335

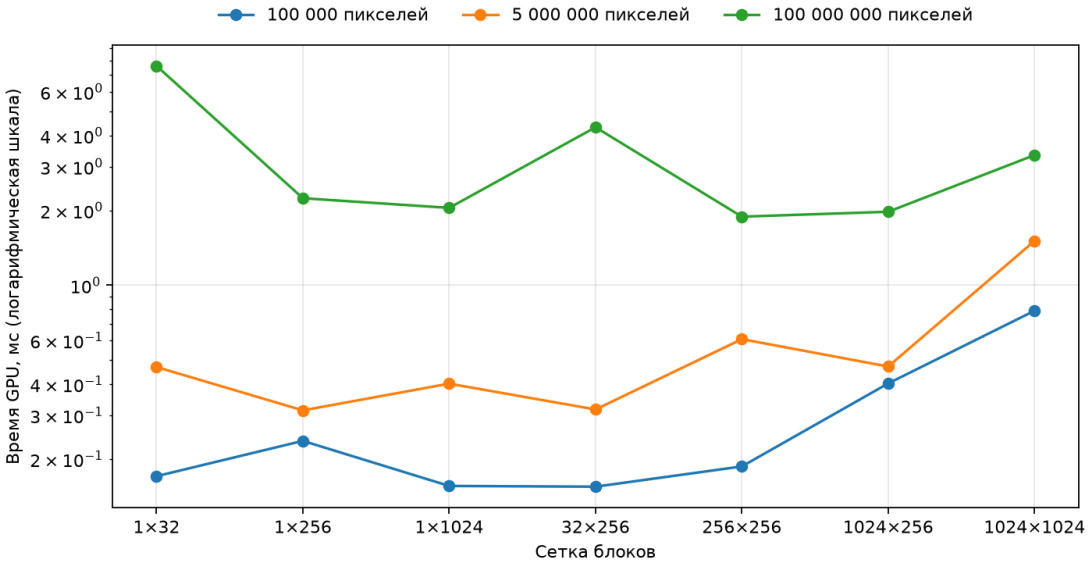


Рис. 1: Время работы GPU при разных конфигурациях сетки

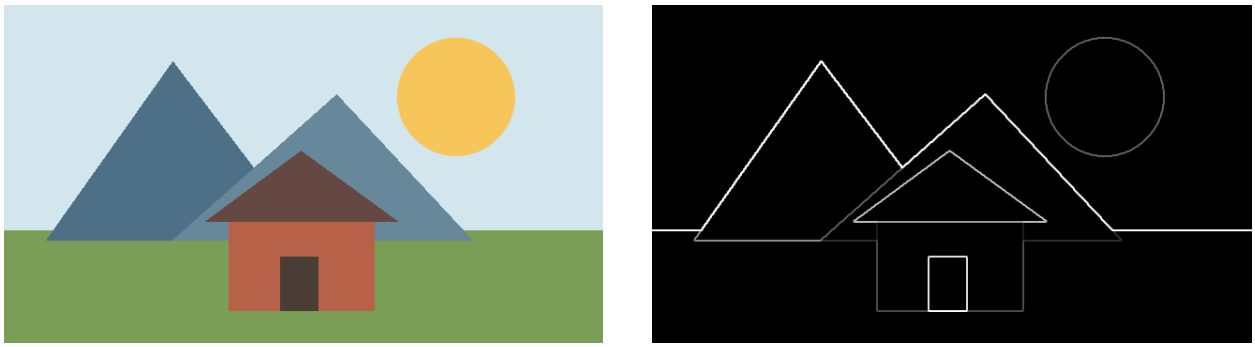


Рис. 2: Синтетическое изображение до обработки и результат выделения контуров

6 Выводы

Метод Собеля выделяет границы по изменению яркости. Его применяют в обработке изображений и компьютерном зрении, например перед анализом форм объектов.

При реализации использованы двумерная сетка потоков и текстурная память. Режим Clamp обеспечивает обработку границ изображения. Основная сложность заключалась в согласовании индексов соседних пикселей и проверке результата на примерах из задания.

На изображении из $100\,000\,000$ пикселей при сетке $(256, 256)$ однопоточная CPU-реализация затратила 1024,48 мс, а ядро GPU — 1,887 мс. Эти интервалы охватывают разные этапы: в измерение GPU не входит передача данных. Замеры выполнены на одном запуске.